0/5 Questions Answered

Project initial implementation - serving team member

Q1 Joint responsibilities

1 Point

Please upload:

A pair of JSON files representing

- one input sample (with real representative values)
- and one model output (again, with real representative values).

(Training, serving, and data team members must agree on this item together; training and inference team members ingest an input like the sample and produce an output like the sample, data team member produces something like the input sample in online and offline workflows.)

If your project involves more than one model, you will have one JSON pair per model. In that case, please name each JSON clearly to indicate which model it "belongs" to.

Input JSON sample(s):

Please select file(s) Select file(s)

Output JSON samples(s):

Please select file(s) Select file(s)

(4-person team only) If you are part of a four-person team, also upload

- a table enumerating all the containers involved in each role, with links to their Dockerfiles/Docker Compose files, and a link to the equivalent K8S manifest for each one.

The object is to show that each of the individual role-owned systems will be supported by the DevOps/Platform role, although at this stage team members are working independently.

> **Upload in PDF format.**

Please select file(s) Select file(s)

Save Answer

Q2 Serving contributions

9 Points

The serving role must prepare a set of measured serving options (fast/good/cheap tradeoffs) so the team can choose a deployment approach during system integration.

Requirements to get credit for those deliverables:

- All experiments should run on Chameleon, from inside a container on a compute instance. (For purposes of this project, "local" work doesn't count.)
- You must prepare and evaluate a variety of serving options, including a baseline option and some optimized options. Your optimizations should include model-level, system-level, and infrastructure-level optimizations, separately and in combination. Your evaluations should be appropriate to validate the expected benefit of each optimization, as well as its potential tradeoffs.
- Right-sizing note: for the most promising option(s), clarify CPU/memory (and GPU if any) needs using observed resource usage on Chameleon under a representative load.

Q2.1 Serving options table

4 Points

Prepare a table comparing multiple serving options, with the most promising options clearly marked (best options with respect to different priorities). An example of a few rows of such a table is given below.

Option	Endpoint URL	Model version	Code version	Hardware	p50/p95 latency	Throughput	Error rate	Concurrency tested	Compute instance type	Notes
baseline_http	http://...	model id	sha	CPU	...	...	...	...	cpu/mem	simplest reference
onnx_quant	http://...	model id	sha	CPU	...	...	...	...	cpu/mem	model-level optimization
triton_batch	http://...	model id	sha	GPU or CPU	...	...	...	...	cpu/gpu/mem	system-level optimization

Upload in PDF format.

Please select file(s) Select file(s)

Note: There is no credit for systems that are not running on Chameleon Cloud. Your table should only include the results of experiments you have run on Chameleon instances. Do not include any "local" work.

Save Answer

Q2.2 Repository artifacts

4 Points

Please upload serving artifacts. Make sure to name these carefully so that it is clear how these items are used.

Dockerfile(s) for serving (if your table of experiments involves multiple frameworks, you may have several):

Please select file(s) Select file(s)

Serving code/serving config file(s) depending on framework. ("Serving code" can include scripts that consume a model artifact and product an optimized model artifact, which is then served.)

Please select file(s) Select file(s)

Scripts or notebooks for evaluating a serving configuration:

Please select file(s) Select file(s)

Use this text field to explain which items go together/which table rows come from which artifacts.

Note: There is no credit for systems that are not running on Chameleon Cloud. Code or configuration that has not been executed in the target environment, or that only runs locally, does not count. You should *only* submit items that you have implemented and executed on Chameleon.

Save Answer

Q2.3 Demo video

1 Point

Upload a sped-up demo video: show your most promising serving option running on Chameleon, and responding to the agreed example request(s).

The video should be no longer than a few minutes.

Please select file(s) Select file(s)

Note: There is no credit for systems that are not running on Chameleon Cloud. Code or configuration that has not been executed in the target environment, or that only runs locally, does not count. You should *only* submit items that you have implemented and executed on Chameleon.

Save Answer

Q3 Bonus

0 Points

If you are attempting the extra credit:

Integrate a serving framework not used in the lab (e.g. *not* FastAPI/Triton Inference Server) that improves your serving design in a meaningful way, and justify why it improves your design relative to the lab frameworks, with a concrete, realistic example. (Examples: Ray Serve, KServe, etc.)

Submit a PDF document explaining in detail, including the concrete realistic example, and also any artifacts (e.g. Dockerfiles, Python scripts) that go along with it.

Please select file(s) Select file(s)

Note: There is no credit for systems that are not running on Chameleon Cloud. Code or configuration that has not been executed in the target environment, or that only runs locally, does not count. You should *only* submit items that you have implemented and executed on Chameleon.

Save Answer

Save All Answers

Submit & View Submission